Circular Doubly Linked List – Destroy List –Space Complexity and Algorithm

Program:

```
void destroyList()
{
    if (head == nullptr)
    {

        cout << "List is empty." << endl;
        return;
    }

    Node *current = head;
    Node *nextNode;

    // --- CDLL Change ---
    // Traverse and free nodes, stopping when we return to
the start
    do
    {
        nextNode = current->next;
        free(current);
        current = nextNode;
    } while (current != head);

    head = nullptr;
    cout << "List destroyed." << endl;
}
……
       case 9:
            destroyList();
            break;
```

Space Complexity Analysis

Input Space Complexity

A. Parameter-Only Definition

Function signature:

```
void traverseInOrder()
```

- It takes **no parameters**.
- Hence, input space is constant.

- **Function's Parameter:** None (void).
- **Analysis:** The function accepts no arguments.

Input Space (Parameter-only): O(1)

(Zero parameters = constant space)

B. Full-Data Definition

Under full-data definition, **input includes the entire data structure the function works on**, even if not fully manipulated.

Here, the input includes:

- The global pointer `head`
- The entire circular doubly linked list with **n nodes**

The function:

- Accesses `head`
- Traverses the entire list using `temp = temp->next`
- Reads every node's `data`, `next`, and `prev`

- **Function's Data:**
  - The entire circular doubly linked list referenced by the global `head` pointer.

- **Analysis:** To destroy the list, the function must access every single node. If the list has n nodes, the "input context" is that list of size n.

Thus, the list with **n nodes** is considered part of the input.

Input Space (Full Data): O(n)

(The circular doubly linked list stored in memory occupies linear space)

Auxiliary Space Complexity

Auxiliary space = extra temporary memory used by the algorithm **beyond the input list**.
Auxiliary space refers to the extra or temporary space used by the algorithm during its execution to perform the task.

The function uses:

```
Node *temp = head;
```

A. Local Variables

- `Node *temp` → constant-size pointer → **O(1)**

B. Dynamic Memory

- No `new` / `malloc`
- No extra data structure
- No recursion

C. Loop Uses No Extra Space

The loop:

```
do {
    ...
    temp = temp->next;
} while (temp != head);
```

- Does **not** allocate memory
- Only updates the pointer

Analysis of Code:

- **Stack Memory (Local Variables):**
  - Node *current: A single pointer variable used to track the node being deleted.
  - Node *nextNode: A single pointer variable used to save the reference to the *next* node before the *current* one is deleted (to prevent losing the link).
- **Heap Memory:**
  - **Not** allocating new memory.
  - Calling free(). While this operation modifies the heap by *releasing* memory, the algorithm itself does not require extra storage proportional to the input size to perform the deletion.
- **Iterative vs. Recursive:**
  - Since used a do-while loop (iterative), the stack memory usage is constant.
  - *Note:* If used recursion to destroy the list, the auxiliary space would have been O(n) due to the function call stack. But here, it is efficient.

Conclusion:

- The function uses exactly two pointers (current and nextNode) regardless of the size of the list.
- Whether the list contains 10 nodes or 10 million nodes (n), the temporary space needed to manage the deletion process is constant.

Auxiliary Space Complexity: O(1) (Constant space)

(The function uses only one pointer regardless of list size)

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\underbrace{Space}_{} = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(1)}_{\text{Input}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\underbrace{Space}_{} = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(n)}_{\text{Input}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity (O(1))** is the expected answer.

Algorithm: Destroy CircularDoubly Linked List

CDDESTROY(START)

1. [***CHECK IF LIST IS EMPTY***]
If START = NULL, then:
Write: "List is empty." and EXIT.
[*End of If structure*]

2. [***INITIALIZE TRAVERSAL POINTER***]
Set PTR := START. [*Note: We need to remember the original HEAD/START address to know when to stop, even though we are about to free it.*]

3. [***DELETION LOOP***]
Repeat:

a. [***SAVE NEXT NODE***]
Set NEXTNODE := NEXTLINK[PTR]. [*Save reference to the next node before deleting current*]

b. [***FREE MEMORY***]
Free PTR. [*Return the current node(PTR) to memory bank*]

c. [***MOVE FORWARD***]
Set PTR := NEXTNODE. [*Move to the next node*]

Until PTR = START. [*Stop when we wrap around to the address where we started*]

4. [***RESET HEAD POINTER***]
Set START := NULL. [*Ensure the global pointer doesn't point to freed memory*]

5. [***PRINT MESSAGE***]
Write: "List destroyed."

6. ***Exit***
